

One-to-One and One-to-Many Relationships

- Due No Due Date
- Points 1
- Submitting a website url

Learning Goals

- Use SQLAlchemy to join tables with one-to-one, one-to-many, and many-to-many relationships.

Introduction

We already know that we can build our SQL tables such that they associate with one another via **primary keys** and **foreign keys**. We can also use SQLAlchemy to access data across different tables by establishing relationships in code, without having to write tons of code ourselves, following the idea of **convention over configuration**.

SQLAlchemy relationships make it easy to establish relationships between our models, without having to write a ton of SQL ourselves. Sounds great, right? Now that we have you totally hooked, let's take a look at how we use these SQLAlchemy relationships.

Before we begin, run `pipenv install && pipenv shell` to generate and enter your virtual environment. This will install `sqlalchemy`, `alembic`, `faker`, `pytest`, and `ipdb`.

How do we use SQLAlchemy Relationships?

SQLAlchemy ORM uses a `ForeignKey` column to constrain and join data models, as well as a `relationship()` method that provides a property to one model that can be used to access its related model. SQLAlchemy ORM also uses a `backref()` method to create the property in the related

model. This makes the syntax for accessing related models and creating join tables very simple.

One-to-Many Example

Here is some code that you might use to create a relationship between an order and a customer:

```
# example
```

```
class Customer(Base):
    __tablename__ = 'customers'

    id = Column(Integer(), primary_key=True)

    orders = relationship('Order', backref='customer')

class Order(Base):
    __tablename__ = 'orders'

    id = Column(Integer(), primary_key=True)
    customer_id = Column(Integer(), ForeignKey('customers.id'))
```

We can see that the `Customer` model contains an `orders` class attribute that is set by the `relationship()` method. The `backref()` method therein sets the reverse many-to-one relationship that **refers back** from the `Order` model. Inclusion of a foreign key for a `Customer` instance in the `Order` model is all that is needed to complete this relationship.

(If you don't need think you'll need the many-to-one relationship, you can leave out the `backref` .)

One-to-One Example

One-to-one relationships aren't *tremendously* common, but they're simple to build in SQLAlchemy ORM if you ever need to!

Let's say that orders have a lot of important associated data, but only two or three columns' worth of regularly-queried data. In designing a database to return data quickly, it makes the most sense for us to separate the important attributes into a smaller table and create a one-to-one relationship with the beefier database for metadata.

example

```
class Order(Base):
    __tablename__ = 'orders'

    id = Column(Integer(), primary_key=True)

class OrderMetadata(Base):
    __tablename__ = 'orders_metadata'

    id = Column(Integer(), primary_key=True)
    order_id = Column(Integer(), ForeignKey("order.id"))

    order = relationship('Order',
        backref=backref('order_metadata', uselist=False))
```

The syntax for a one-to-one relationship is identical to that of a one-to-many relationship, with the exception that the `backref()` method takes `uselist=False` as an optional second parameter. This means exactly what it sounds like: the `Order` model refers back to the `OrderMetadata` model via a property `order_metadata` that is not a list. (If it were a list, it would be many!)

Overview

In this lesson, we'll be building out a **one-to-many** relationship between two models: **games** and **reviews**. We'll set up our database so that a game **has many** reviews, and each review **belongs to** a specific game.

By writing a few migrations and making use of the appropriate SQLAlchemy methods, we'll be able to:

- Ask a game about its reviews.
- Ask a review about its game.

Here's what our Entity Relationship Diagram (ERD) looks like:



Building our Migrations

The Game Model

A game will *have many* reviews. Before we worry about the migration that will implement this in our reviews table, let's think about what that table will look like:

id	title	platform	platform	price
1	Breath of the Wild	Switch	Action-adventure	60

Our games table doesn't need any information about the reviews, so it makes sense to generate this table first: it doesn't have any dependencies on another table. This makes sense even thinking about our domain in the real world: a game can exist without any reviews.

The basic structure for your SQLAlchemy app is already configured. Navigate to the `one_to_many` directory and you should see the following directory structure:

```
.
├─ alembic.ini
├─ debug.py
├─ migrations
│   └─ README
│   └─ env.py
│   └─ script.py.mako
│   └─ versions
│       └─ 0d06d41c7860_initialize_database.py
│       └─ 461383e0c920_create_tables_games_reviews.py
├─ models.py
├─ seed.py
└─ testing
    └─ conftest.py
    └─ test.py
    └─ test.py
```

`alembic.ini` points to a SQLite database called `one_to_many.db`. The models should go into `models.py`; it is already configured to create a `Base`, and `env.py` is pointing to its metadata.

Run `alembic upgrade head` to create your database.

Navigate to `app/db.py` and build a basic model for the `games` table:

```
# models.py
```

```
class Game(Base):
    __tablename__ = 'games'

    id = Column(Integer(), primary_key=True)
    title = Column(String())
    genre = Column(String())
    platform = Column(String())
    price = Column(Integer())
```

Run `alembic revision --autogenerate -m'Create Game Model'` from inside of the `one_to_many` directory. You should see the following output:

```
INFO [alembic.runtime.migration] Context impl SQLiteImpl.
INFO [alembic.runtime.migration] Will assume non-transactional DDL.
INFO [alembic.autogenerate.compare] Detected added table 'games'
Generating ...python-p3-sqlalchemy-one-to-many/one-to-many/migrations/versions/62797912f786_create_model.py ... done
```

Run `alembic upgrade head` to generate a database with your `Game` model.

The Review Model

A review will **belong to** a specific game. What does that mean in terms of our database? Think back to what you learned about SQL and joining between multiple tables. How can we connect between a review and its associated game?

That's right, we need a **foreign key**! Since a review belongs to a specific game, we need some way of indicating on the review *which* specific game it belongs to.

Let's take a look at what our `reviews` table will need to look like:

id	score	comment	id
1	10	A classic!	1

Ok! Now that we know what we need to create, let's head back to `models.py` and write out a new model:

```
# models.py
```

```
class Review(Base):
    __tablename__ = 'reviews'

    id = Column(Integer(), primary_key=True)
    score = Column(Integer())
    comment = Column(String())
    game_id = Column(Integer(), ForeignKey('games.id'))
```

Make sure to add the relationship to `Game` as well:

```
# models.py
```

```
class Game(Base):

    # tablename, columns

    reviews = relationship('Review', backref=backref('game'))
```

Lastly, don't forget to set the `__repr__()` for each of your models. You'll thank me later!

```
# models.py
```

```
class Game(Base):
```

```
# tablename, columns, relationship

def __repr__(self):
    return f'Game(id={self.id}, ' + \
           f'title={self.title}, ' + \
           f'platform={self.platform})'

class Review(Base):

    # tablename, columns, relationship

    def __repr__(self):
        return f'Review(id={self.id}, ' + \
               f'score={self.score}, ' + \
               f'game_id={self.game_id})'
```

Great! Now go ahead and run the same commands in your terminal to generate and run our migrations:

```
$ alembic revision --autogenerate -m'Add Review Model'
# => INFO [alembic.runtime.migration] Context impl SQLiteImpl.
# => INFO [alembic.runtime.migration] Will assume non-transactional DDL.
# => INFO [alembic.autogenerate.compare] Detected added table 'reviews'
# => Generating ...sqlalchemy-relationships/python-p3-sqlalchemy-one-to-many/one_to_many/migrations/versions/1f2ce6b597d

$ alembic upgrade head
# => INFO [alembic.runtime.migration] Context impl SQLiteImpl.
# => INFO [alembic.runtime.migration] Will assume non-transactional DDL.
# => INFO [alembic.runtime.migration] Running upgrade 62797912f786 -> 1f2ce6b5977d, Add Review Model
```

There is also some code in the `seed.py` file that we'll use to generate some data for our two models. In the seed file, we first create a game instance, then use the ID from that game instance to associate it with the corresponding review. The data itself is generated automatically using

`faker` .

Run this to fill the database with games and reviews:

```
$ python seed.py
```

Exploring our Relationships Using SQLAlchemy

A Review Belongs to a Game

Let's enter the Python shell and access the first review. Run `python debug.py` to set up a session and import the data models:

```
# Access the first review instance in the database
review = session.query(Review).first()
review
# => Review(id=1, score=7, game_id=1)

# Get the game_id foreign key for the review instance
review.game_id
# => 1
```

We know that this review has some relationship to data in the `games` table. We could even use the foreign key to access that data directly:

```
# Find a specific game instance using an ID
session.query(Game).filter_by(id=review.game_id).first()
# => Game(id=1, title=Jacob Floyd, platform=wii u)
```

But SQLAlchemy provides an even easier route to the related game:

```
review.game
```

```
# => Game(id=1, title=Jacob Floyd, platform=wii u)
```

Nice!

A Game Has Many Reviews

Now let's access the first game:

```
game = session.query(Game).first()
game
# => Game(id=1, title=Jacob Floyd, platform=wii u)
```

As expected, this is the same game that we found through the first review. We can search for reviews using this game's ID as a filter:

```
reviews = session.query(Review).filter_by(game_id=game.id)
[review for review in reviews]
# => [Review(id=1, score=7, id=1), Review(id=2, score=7, id=1), Review(id=3, score=8, id=1)]
```

But just as with finding the game for a review, we can access a game's reviews directly using its relationship:

```
game.reviews
# => [Review(id=1, score=7, game_id=1), Review(id=2, score=7, game_id=1), Review(id=3, score=8, game_id=1)]
```

Conclusion

In this lesson, we explored the most common kind of relationship between two models: the **one-to-many** or "has-many"/"belongs-to" relationship. With a solid understanding of how to connect databases using primary and foreign keys, we can take advantage of some helpful SQLAlchemy methods that make it much easier to build comprehensive database schemas and integrate them into our Python applications.

Run `pytest -x` to make sure all of the tests are passing. In the next lesson, we'll explore many-to-many relationships in SQLAlchemy

Solution Code

```
from sqlalchemy import create_engine
from sqlalchemy import ForeignKey, Column, Integer, String
from sqlalchemy.orm import relationship, backref
from sqlalchemy.ext.declarative import declarative_base

engine = create_engine('sqlite:///one_to_many.db')

Base = declarative_base()

class Game(Base):
    __tablename__ = 'games'

    id = Column(Integer(), primary_key=True)
    title = Column(String())
    genre = Column(String())
    platform = Column(String())
    price = Column(Integer())

    reviews = relationship('Review', backref=backref('game'))

    def __repr__(self):
        return f'Game(id={self.id}, ' + \
            f'title={self.title}, ' + \
            f'platform={self.platform})'

class Review(Base):
    __tablename__ = 'reviews'

    id = Column(Integer(), primary_key=True)
    score = Column(Integer())
```

```
comment = Column(String())

game_id = Column(Integer(), ForeignKey('games.id'))

def __repr__(self):
    return f'Review(id={self.id}, ' + \
        f'score={self.score}, ' + \
        f'game_id={self.game_id})'
```

Resources

- [Python 3.8.13 Documentation](https://docs.python.org/3/) ➞ (https://docs.python.org/3/)
- [SQLAlchemy ORM Documentation](https://docs.sqlalchemy.org/en/14/orm/) ➞ (https://docs.sqlalchemy.org/en/14/orm/)
- [Alembic 1.8.1 Documentation](https://alembic.sqlalchemy.org/en/latest/) ➞ (https://alembic.sqlalchemy.org/en/latest/)
- [Basic Relationship Patterns - SQLAlchemy](https://docs.sqlalchemy.org/en/14/orm/basic_relationships.html) ➞ (https://docs.sqlalchemy.org/en/14/orm/basic_relationships.html)